Implement Linear Regression from Scratch using Gradient Descent

Instructions:

1. Setup Your Environment

- Create a new Python notebook on your local machine (Jupyter, Colab, etc.).
- Make sure you have **Python** installed (version 3.x).
- Import the necessary libraries: `numpy` for computations and `matplotlib` for plotting.

2. Generate Synthetic Data

- Create or import a dataset with a simple linear relationship:
  - Define the number of samples (e.g., 100).
  - Create feature values (`X`) randomly.
  - Define true parameters (`theta_true`) and generate target values (`y`) with some noise.
- Visualize the data points using a scatter plot.

3. Define the Hypothesis Function

- Write a function `hypothesis(X, theta)` that calculates predicted values:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

4. Implement the Cost Function

- Implement the Mean Squared Error (MSE) cost function:

$$J(w,b) = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$$

- This function will help us evaluate how well the model fits the data.

5. Calculate the Gradient

- Derive and implement the gradient of the cost function with respect to each parameter:

For each weight w_j:

$$\frac{\partial J}{\partial w_j} = \frac{1}{m}\sum_{i=1}^{m} \frac{\partial J}{\partial z^{(i)}} \frac{\partial z^{(i)}}{\partial w_j} = \frac{1}{m}\sum_{i=1}^{m} \left(\hat{y}^{(i)} - y^{(i)}\right) x_j^{(i)}$$

For the bias b:

$$\frac{\partial J}{\partial b} = \frac{1}{m}\sum_{i=1}^{m} \frac{\partial J}{\partial z^{(i)}} \frac{\partial z^{(i)}}{\partial b} = \frac{1}{m}\sum_{i=1}^{m} \left(\hat{y}^{(i)} - y^{(i)}\right)$$

6. Perform Gradient Descent

- Initialize parameters (`theta`) with zeros or random values.
- Set the learning rate (`alpha`), e.g., 0.01.
- Iteratively update parameters:

$$w \leftarrow w - \alpha \cdot \frac{1}{m} X^{\top}(\hat{y} - y), \qquad b \leftarrow b - \alpha \cdot \frac{1}{m}\sum_{i=1}^{m}(\hat{y}^{(i)} - y^{(i)})$$

- Run for a fixed number of iterations (e.g., 1000).
- Track and plot the cost value over iterations to visualize convergence.

7. Evaluate Your Model

- After training, print the final parameters.
- Plot the fitted line against the original data points.
- Optionally, test your model on new data.